

SubEco — 발표 대본 & 예상 질문지

구독 관리 트래커 · Spring Boot 4 학습 프로젝트 · 2026

SubEco 발표 대본

구독 관리 트래커 · Spring Boot 4 학습 프로젝트 (1~14강)

발표 7~10분 + 질의응답 5분 / 팀원 5명 전원 발표

슬라이드 번호는 [docs/index.html](#) 텍 기준. 각 절 끝의 ▶ [루브릭](#) 은 어떤 채점 항목을 노리는지 표시.

발표 구성 & 타이밍 (총 ≈ 9분)

순서	발표자	담당	슬라이드	시간
1	이태경	서비스 기획·비즈니스 · 도메인 모델 · 예외처리 · 댓글 엔티티	1~3, 댓글	2:15
2	김세진	구독 CRUD (Service-Controller-PRG) · REST API	4~5, REST	2:15
3	이상윤	환율 서비스 · 문제 해결 · 테스트 코드	6~7, 테스트	2:15
4	김준아	SubscriptionView DTO · 홈 대시보드	8~9	1:15
5	이예은	UI·UX 디자인 · Mustache 뷰 · 마무리	10~12	1:30

시간이 빠듯하면 각자 굵게 표시된 **핵심 문장**만 말하고 카드 세부는 생략. 발표 전달력 10점은 *전원 참여 + 시간 준수*가 핵심.

1. 이태경 — 기획·비즈니스 · 도메인 · 예외처리 · 댓글

[슬라이드 1 — 타이틀]

"안녕하세요, 저희 팀 프로젝트 **SubEco** — **구독 관리 트래커**를 발표하겠습니다. 저는 기획과 도메인 설계를 맡은 이태경입니다."

[슬라이드 2 — 서비스 소개 + 도메인]

"한 줄로 말하면, **흘어진 구독 서비스의 결제일과 금액을 한 곳에서 관리해 주는 가계부형 대시보드**입니다."

- **타겟:** Netflix·Spotify·ChatGPT처럼 여러 구독을 동시에 쓰는 2030 사용자입니다. '내가 지금 몇 개를 결제 중이고, 다음 결제가 언제인지' 모르는 사람이 대상입니다.
- **수업 예제(Article 게시판)와의 차별점:** 단순 글 CRUD가 아니라, **결제 주기로 다음 갱신일을 자동 계산**하고, **외화 구독을 실시간 환율로 원화 환산**하며, **갱신 임박을 알려 주는 도메인 로직**이 핵심입니다.
- **비즈니스 가능성:** 갱신 임박 알림에 기반한 **광고·제휴 추천**(더 싼 요금제 안내), 안 쓰는 구독 정리로 비용을 줄여 주는 **구독 다이어트 코칭** 모델로 확장할 수 있습니다. 개인 자산관리 앱의 한 기능으로 붙이는 방향도 현실적입니다.

도메인은 **Subscription** 엔티티가 중심이고, 결제주기·통화·분류 태그를 **enum**으로 두었습니다. 특히 **BillingCycle** enum은 **addTo()** 메서드로 **날짜 계산 로직을 enum 안에 내장**해, 월/연 갱신 계산이 한 곳에 모이도록 설계했습니다."

[슬라이드 3 — 예외 처리]

"존재하지 않는 구독에 접근하면 **SubscriptionNotFoundException** 을 던지고, **@ControllerAdvice** 인 **GlobalExceptionHandler**가 이를 받아 **404 상태 + 사용자용 에러 페이지**로 변환합니다. 잘못된 요청에 빈 화면이나 스택트레이스 대신 **친절한 안내 페이지**를 보여 주는 UX 배려입니다."

[슬라이드 — 댓글 엔티티]

"마지막으로 14장 범위인 **댓글(메모) 기능**의 토대를 만들었습니다. **Comment** 엔티티가 구독을 **단방향 @ManyToOne**으로 참조하고, **CommentRepository**의 **findBySubscriptionId**로 구독별 메모를 조회합니다. 양방향 대신 단방향을 택해 엔티티를 단순하게 유지했습니다.

다음은 CRUD와 REST API를 맡은 김세진입니다."

▶ 루브릭: 서비스 독창성·기획력(12), 비즈니스 가능성(8), 디테일·UX(10) 일부, 소스흐름 보고서(20)(어노테이션 역할 설명)

2. 김세진 — 구독 CRUD · REST API

[슬라이드 4 — SubscriptionService]

"CRUD를 맡은 김세진입니다. 비즈니스 로직은 **SubscriptionService**에 모았습니다. 생성·조회·수정·삭제 **네 가지가 모두 동작**하며,

- 클래스에는 기본 **@Transactional(readOnly = true)**를 걸고, **변경 메서드만 @Transactional**로 덮어썼습니다.
- 수정은 **save()** 호출 없이 **더티 체킹**으로 반영됩니다. 트랜잭션 안에서 엔티티 필드만 바꾸면 JPA가 변경을 감지해 UPDATE를 날립니다.
- 조회 시점에 **지난 갱신일을 오늘 이후까지 자동으로 밀어 주는 보정 로직**도 서비스에 있습니다."

[슬라이드 5 — Controller · Form · PRG]

"화면용 컨트롤러 **SubscriptionController**는 7개 URL로 목록·등록·상세·수정·삭제를 라우팅합니다.

- 입력은 엔티티가 아니라 `SubscriptionForm` DTO로 받아 `toEntity()` 로 변환합니다. 엔티티를 직접 폼에 노출하지 않는 안전한 구조입니다.
- 저장·수정·삭제 같은 POST 처리 뒤에는 **PRG 패턴**으로 `redirect:` 시켜, 새로고침 시 **중복 저장을 방지**합니다."

[슬라이드 — REST API]

"여기에 더해 **REST API**를 별도로 구현했습니다. `@RestController` 인 `SubscriptionApiController` 가 `/api/subscriptions` 경로로 **GET·POST·PATCH·DELETE 5개 엔드포인트**를 제공하고, 반환 객체는 Jackson이 **JSON**으로 직렬화합니다.

중요한 점은, 화면용 404는 HTML 페이지지만 **REST는 컨트롤러 로컬 `@ExceptionHandler` 로 `{"message": ...}` JSON 404**를 따로 응답한다는 겁니다. 같은 예외라도 클라이언트에 맞는 형식으로 돌려줍니다. (시연/캡처: `GET /api/subscriptions`, 없는 `id` 404 JSON)

다음은 환율 연동과 테스트를 맡은 이상윤입니다."

▶ 루브릭: **CRUD 기능 완성도(20), REST API/JSON(10)** — `@RestController` 2개 이상·JSON 정상·캡처 포함

3. 이상윤 — 환율 서비스 · 문제 해결 · 테스트

[슬라이드 6 — ExchangeRateService]

"환율과 테스트를 맡은 이상윤입니다. 외화 구독을 원화로 보여 주려고 **외부 환율 API**를 연동했습니다.

- 매 요청마다 호출하면 느리니, 앱 시작 시 `@PostConstruct` 로 한 번 불러와 `ConcurrentHashMap` 에 캐싱하고, `@Scheduled` cron으로 **매일 자정 갱신**합니다.
- API가 실패해도 `Currency` enum의 폴백 환율로 앱이 계속 동작합니다. JPY는 직접 환율 대신 **USD 기준 교차 환율**로 계산합니다."

[슬라이드 7 — 문제 해결]

"개발 중 실제로 막혔던 문제 두 가지입니다.

1. 처음 쓴 **Frankfurter API가 KRW를 지원하지 않아** JSON 대신 HTML 에러를 반환했습니다. 네트워크 문제인 줄 알고 핫스팟까지 바꿔 봤지만, 원인은 **ECB 데이터에 원화가 없어서**였고, **KRW를 지원하는 open.er-api.com으로 교체**해 해결했습니다.
2. Spring Boot 4 환경에서 `RestClient.Builder` 빈 자동 등록과 커스텀 record 역직렬화가 말썽이라, 응답을 `Map<String, Object>` 로 받는 방식으로 바꿔 의존성 문제를 한 번에 없앴습니다."

[슬라이드 — 테스트 코드]

"품질 확인을 위해 **테스트 9개**를 작성했고 전부 통과합니다. **MockMvc로 REST API 통합 테스트 6개**(목록·단건·404·생성·수정·삭제), **댓글 리파지터리 테스트 2개**, **컨텍스트 로드 1개**입니다.

테스트 작성 중에도 문제가 있었는데, **Spring Boot 4는 Jackson 3와 모듈형 스타터를 써서 ObjectMapper**가 **tools.jackson** 패키지로, **@AutoConfigureMockMvc**도 경로가 바뀌어 있었습니다. 추측 대신 **클래스패스 jar를 직접 열어 실제 패키지를 확인**해 import를 교정했습니다.

다음은 화면 데이터 가공을 맡은 김준아입니다."

▶ 루브릭: **소스흐름 보고서(20)** — 실제 오류 2건 이상 해결 과정 충족, **REST API/JSON(10)** 신뢰성 보강

4. 김준아 — SubscriptionView DTO · 홈 대시보드

[슬라이드 8 — SubscriptionView]

"데이터 가공을 맡은 김준아입니다. Mustache 템플릿은 로직을 못 쓰기 때문에, **화면에 필요한 모든 계산을 SubscriptionView 생성자에서 끝냅니다.**

displayPrice (외화면 원화 병기), **daysText** ('오늘' / 'N일 후'), **dueSoon** (임박 배지), **colorClass** (카드 색상), **emoji** (서비스 아이콘), **monthlyKrw** (연간이면 ÷12) 등을 미리 만들어 두고, 템플릿은 **출력만** 합니다. 불변 객체라 안전하고 디버깅도 쉽습니다."

[슬라이드 9 — 홈 대시보드]

"홈은 **HomeController**가 구독 전체를 모아 **월 합산 지출**, **이번 달 갱신 건수**, **7일 이내 임박 건수**, **자동결제 개수**를 집계하고, **갱신일 순으로 정렬해 임박 4건**을 보여 줍니다. '내 구독을 한눈에'라는 기획 의도를 그대로 화면에 옮긴 부분입니다.

다음은 UI와 뷰를 맡은 이예은입니다."

▶ 루브릭: **디테일·UX(10)**, **소스흐름 보고서(20)** (계층 분리 설명)

5. 이예은 — UI·UX · Mustache · 마무리

[슬라이드 10 — UI·디자인]

"UI를 맡은 이예은입니다. 다크 테마에 **CSS 변수로 색상을 일원화**하고, 카드 상단 색 스트라이프와 서비스별 이모지로 한눈에 구분되게 했습니다.

사용자 배려(UX)도 신경 썼습니다. 구독이 하나도 없으면 **빈 목록 안내**를 보여 주고, 잘못된 주소로 들어오면 **커스텀 404 페이지**로 안내하며, 갱신이 임박하면 카드에 **강조 배지**를 띄웁니다."

[슬라이드 11 — Mustache 템플릿]

"뷰는 **Mustache 서버사이드 렌더링**입니다. 목록·등록·수정·상세·환율·에러 페이지를 템플릿으로 나누고, 공통 헤더/푸터는 **파셜로 재사용**합니다. 로직이 없는 템플릿이라, 모든 계산은 앞서 본

SubscriptionView가 담당합니다."

[슬라이드 12 — 마무리]

"정리하면, 저희는 1~14강 범위의 **MVC·JPA·CRUD·REST API·JSON**을 실제 구독 관리라는 주제로 구현했고, **자동 갱신일 계산·실시간 환율·대시보드**로 수업 예제와 차별화했습니다. 들어 주셔서 감사합니다. 질문 받겠습니다."

▶ 루브릭: **디테일·UX(10)**(빈 목록·오류 안내), **발표 전달력(10)**(전원 참여·마무리)

발표 전 체크리스트

- ☐ bootRun 띄워 두고 **REST API JSON 응답·404 화면을 실제 시연**(또는 캡처 슬라이드 준비) — REST/JSON 항목은 **캡처 포함**이 채점 기준
- ☐ 보고서(PDF)에 **계층 흐름 도식 + 어노테이션 역할 + 오류 2건 해결**이 들어갔는지 확인
- ☐ 각자 **자기 담당 코드 파일명·메서드명**을 말할 수 있게 (질의응답 개인별 10점)
- ☐ 7~10분 안에 끝나도록 **1회 리허설**

SubEco 예상 질의응답 (Q&A 대비)

질의응답은 **개인별 10점** — "담당 코드 질문에 정확히 답변 + 응용 질문 대응"이 만점 기준.
각자 **자기 담당 파트** 답변은 외우고, 공통/응용 질문은 팀 전체가 한 번씩 읽어 둘 것.
답변은 길게 말하지 말고 **핵심 한두 문장** → 필요시 부연 순서로.

A. 이태경 — 기획·도메인·예외·댓글

Q1. 수업 예제(Article 게시판)와 무엇이 본질적으로 다른가요?

A. 글을 저장·조회하는 단순 CRUD가 아니라, **결제주기로 다음 갱신일을 계산하고** (`BillingCycle.addTo`), **외화를 실시간 환율로 환산하고**, 갱신 임박을 판별하는 **도메인 로직**이 핵심입니다. 데이터를 보여 주는 게 아니라 "언제.얼마가 빠져나가는지"를 가공해 주는 서비스입니다.

Q2. 타깃 사용자와 비즈니스 모델은?

A. 구독을 여러 개 쓰는 2030이 타깃입니다. 갱신 임박 시점에 **더 싼 요금제·대체 서비스**를 추천하는 **제휴 광고**, 안 쓰는 구독 정리 **코칭**이 수익 모델이고, 가계부 앱의 한 기능으로 확장할 수 있습니다.

Q3. `BillingCycle`을 enum으로 만들고 안에 `addTo()` 메서드를 둔 이유는?

A. 결제주기별 날짜 계산이 여러 곳에 흩어지면 버그가 생기기 쉽습니다. **enum 안에 계산을 캡슐화** 하면 분기·반기 같은 주기가 추가돼도 enum 케이스와 메서드만 고치면 됩니다. (`switch` 표현식 사용)

Q4. `@ControllerAdvice`와 `@ExceptionHandler`는 무슨 역할인가요?

A. `@ControllerAdvice`는 모든 컨트롤러에 공통 적용되는 전역 예외 처리기입니다.
`@ExceptionHandler(SubscriptionNotFoundException.class)`로 특정 예외를 잡아,
`@ResponseStatus(404)`와 함께 에러 뷰로 변환합니다.

Q5. 댓글을 양방향이라 아니라 단방향 `@ManyToOne`으로 한 이유는? (응용)

A. 구독 쪽에 `List<Comment>`를 두면 양쪽을 동기화해야 하고 엔티티가 무거워집니다. 조회는 `CommentRepository.findById(id)` 한 줄로 해결되므로, 단방향으로 단순하게 유지했습니다.

Q6. (응용) `@ManyToOne`은 기본 fetch 전략이 뭔가요?

A. **EAGER**입니다. 댓글을 조회하면 연관된 구독도 즉시 로딩됩니다. 목록에서 N+1이 우려되면 fetch join이나 `@BatchSize`로 최적화할 수 있습니다.

B. 김세진 — CRUD · REST API

Q1. `@Controller`와 `@RestController`의 차이는?

A. `@Controller`는 뷰 이름을 반환해 템플릿을 렌더링합니다. `@RestController`는 `@Controller + @ResponseBody`라, 반환 객체를 **JSON 등 응답 본문으로 직렬화**합니다. 그래서 화면용 `SubscriptionController`와 API용 `SubscriptionApiController`를 분리했습니다.

Q2. 수정(update)에서 save() 를 안 부르는데 어떻게 DB에 반영되나요?

A. 더티 체크(변경 감지) 입니다. @Transactional 안에서 영속 상태 엔티티의 필드를 바꾸면, 트랜잭션 커밋 시 JPA가 변경을 감지해 자동으로 UPDATE를 실행합니다.

Q3. @Transactional(readOnly = true) 를 클래스에 걸고, 일부 메서드는 왜 다시

@Transactional 인가요?

A. 조회가 많아 기본은 읽기 전용으로 두어 성능 이점을 얻고, 쓰기가 필요한 메서드만 일반 트랜잭션으로 덮어씁니다. (참고: findAll / findById 도 조회 시 갱신일을 보장·저장하므로 쓰기 트랜잭션을 씁니다.)

Q4. 엔티티를 직접 안 받고 SubscriptionForm · SubscriptionView DTO를 쓰는 이유는?

A. 폼 입력과 화면 출력은 엔티티와 필요한 필드가 다릅니다. DTO로 분리하면 엔티티 내부를 외부에 노출하지 않고, 화면용 계산 필드(displayPrice 등)를 엔티티를 더럽히지 않고 담을 수 있습니다.

Q5. PRG 패턴이 뭐고 왜 쓰나요?

A. POST → Redirect → GET입니다. POST로 저장한 뒤 곧장 화면을 그리면, 새로고침 시 폼이 재전송돼 중복 저장됩니다. 저장 후 redirect: 로 GET을 유도해 이를 막습니다.

Q6. REST API 엔드포인트와 응답 코드를 설명해 보세요.

A. /api/subscriptions 기준 — 목록·단건 GET(200), 생성 POST(201), 수정 PATCH(200), 삭제 DELETE(204) 입니다. 없는 id는 404 JSON으로 응답합니다.

Q7. (응용) JSON 변환은 누가 하나요? SubscriptionView 가 어떻게 JSON이 되죠?

A. Spring이 내장한 Jackson이 getter를 읽어 직렬화합니다. SubscriptionView 는 Lombok @Getter 라 모든 필드가 JSON 키가 됩니다.

Q8. (응용) PATCH인데 전체 필드를 다 바꾸네요? 진짜 PATCH 의미와 다르지 않나요?

A. 맞습니다. 엄밀한 PATCH는 부분 수정이지만, 저희는 기존 화면 수정과 동일하게 전체 교체 시맨틱으로 구현했습니다. 부분 수정으로 바꾸려면 폼에서 null이 아닌 필드만 반영하도록 서비스에서 분기하면 됩니다.

C. 이상윤 — 환율 · 문제해결 · 테스트

Q1. 매 요청마다 환율 API를 부르지 않는 구조를 설명해 보세요.

A. 시작 시 @PostConstruct 로 한 번 호출해 ConcurrentHashMap 에 캐싱하고, @Scheduled(cron = "0 0 0 *") 로 매일 자정 갱신합니다. 화면은 항상 캐시 값을 읽어 빠릅니다.

Q2. API가 죽으면 화면도 죽나요?

A. 아닙니다. 호출을 try/catch로 감싸 실패 시 Currency enum에 박아 둔 폴백 환율을 유지합니다. /rates 페이지에서 실시간/폴백 상태를 표시하고 수동 갱신도 됩니다.

Q3. JPY 환율을 "교차 환율"로 계산한다는 게 무슨 뜻인가요?

A. API는 USD 기준 환율을 줍니다. JPY→KRW 직접 환율이 없으니 (KRW/USD) ÷ (JPY/USD) 로 계산해 1엔당 원화를 구합니다.

Q4. `@PostConstruct` 와 `@Scheduled` 는 각각 언제 실행되나요?

A. `@PostConstruct` 는 빈 생성·의존성 주입 직후 1회, `@Scheduled` 는 지정한 cron 시각마다 주기적으로 실행됩니다.

Q5. 테스트는 어떻게 짰나요? `@SpringBootTest` 와 `MockMvc` 역할은?

A. `@SpringBootTest` 로 실제 스프링 컨텍스트 + H2를 띄우고, `MockMvc`로 HTTP 요청을 흉내 내 `/api/subscriptions` 의 상태코드와 `jsonPath` 로 응답 JSON을 검증합니다. 총 9개 전부 통과합니다.

Q6. 가장 까다로웠던 오류 하나를 설명해 보세요. (담당 코드 핵심)

A. Frankfurter API가 **KRW** 미지원이라 **HTML** 에러를 반환한 문제입니다. 네트워크가 아니라 **데이터 소스(ECB)**에 원화가 없는 게 원인이었고, KRW를 지원하는 open.er-api.com으로 교체해 해결했습니다.

Q7. (응용) `@DataJpaTest` 가 아니라 `@SpringBootTest` 를 쓴 이유는?

A. REST 통합 테스트는 컨트롤러·서비스·리파지터리 전 계층과 **data.sql** 시드가 함께 필요해 전체 컨텍스트를 띄웠습니다. 리파지터리만 가볍게 보려면 `@DataJpaTest` 가 더 적합합니다.

D. 김준아 — SubscriptionView · 홈 대시보드

Q1. 왜 계산을 컨트롤러나 템플릿이 아니라 `SubscriptionView` 생성자에서 하나요?

A. **Mustache**는 로직이 없는 템플릿 엔진이라 if-계산을 못 씁니다. 그래서 화면에 필요한 값을 DTO 생성자에서 모두 계산해 두고 템플릿은 출력만 합니다. 로직이 한곳에 모여 테스트·수정이 쉽습니다.

Q2. `monthlyKrw` (월 환산)는 어떻게 계산하나요?

A. 원화로 환산한 금액을 구하고, 결제주기가 **연간이면 12로 나눠** 월 기준으로 통일합니다. 그래야 홈에서 월 합산 지출을 비교할 수 있습니다.

Q3. 홈 대시보드의 지표들은 어디서 나오나요?

A. `HomeController` 가 구독 전체 (`findAll`)를 받아 스트림으로 집계합니다 — 월 합산 (`sum`), 자동 결제 수 (`filter+count`), 이번 달 갱신 건수, 7일 이내 임박 수, 그리고 **갱신일 정렬 후 상위 4건** (`sorted().limit(4)`).

Q4. (응용) `SubscriptionView` 를 불변(**final 필드**)으로 만든 이점은?

A. 한 번 만들면 값이 안 바뀌어 **여러 화면에서 안전하게 공유**되고, 예기치 않은 수정으로 인한 버그가 없습니다.

E. 이예은 — UI·UX · Mustache

Q1. 사용자 배려(UX)를 어디에서 구현했나요? (루브릭 직결)

A. ① 구독이 없을 때 빈 목록 안내 문구, ② 잘못된 주소·없는 구독에 대한 **커스텀 404 페이지**, ③ 갱신 임박 시 카드 **강조 배지**입니다. 오류 상황에서 빈 화면을 두지 않는 걸 원칙으로 했습니다.

Q2. **Mustache**에서 반복과 조건은 어떻게 표현하나요?

A. 반복은 `{{#subscriptions}} ... {{/subscriptions}}`, 조건은 `{{#dueSoon}} ...`

`{{/dueSoon}}` 처럼 섹션 문법으로 합니다. 값 출력은 `{{serviceName}}` 입니다.

Q3. 헤더/푸터 중복은 어떻게 줄였나요?

A. 공통 영역을 파셜(`{{> layouts/header}}`) 로 분리해 모든 페이지에서 포함합니다.

Q4. (응용) 카드 색상·이미지는 어떻게 정해지나요?

A. 디자인 값도 `SubscriptionView` 가 결정합니다. `colorClass` 는 id를 4로 나눈 나머지로 c1~c4를 순환시키고, `emoji` 는 서비스명을 보고 매핑합니다. CSS는 그 클래스에 스타일만 입힙니다.

F. 공통·심화 (누구에게나 날아올 수 있음)

Q. 전체 요청 흐름을 한 문장으로?

A. **Browser → Controller → Service → Repository → DB**로 내려가고, 데이터는 `SubscriptionView`로 가공돼 **Mustache** 뷰로, 또는 **JSON**으로 돌아옵니다.

Q. `@Entity` 에서 setter 없이 `@Builder` 만 쓴 이유는?

A. 아무 데서나 필드를 바꾸지 못하게 막고, 생성은 빌더로, 수정은 의도가 드러나는 `updateDetails()` 메서드로만 하기 위해서입니다.

Q. enum을 `@Enumerated(EnumType.STRING)` 으로 저장한 이유는?

A. `ORDINAL` (순서 숫자)로 저장하면 enum 순서가 바뀔 때 기존 데이터가 깨집니다. 문자열로 저장하면 안전합니다.

Q. 태그는 별도 엔티티가 아니라 어떻게 저장했나요?

A. `@ElementCollection` 으로 `subscription_tags` 조인 테이블에 문자열 enum으로 저장합니다. 태그가 단순 값 집합이라 엔티티까지는 필요 없다고 판단했습니다.

Q. Spring Boot 4 환경에서 특별했던 점은?

A. **Jackson 3**(`tools.jackson` 패키지)과 모듈형 스타터를 써서, 테스트의 `ObjectMapper` · `@AutoConfigureMockMvc` import 경로가 기존과 달랐습니다. 실제 클래스패스를 확인해 맞췄습니다.

Q. (압박) 만약 구독이 수천 개면 지금 구조에 문제가 없나요?

A. `findAll` 후 메모리에서 집계·필터하므로 대량이면 부담입니다. 실제로는 페이징, 쿼리 단계 집계, 태그 fetch 최적화가 필요합니다. 학습 범위(1~14강)에선 단순·명확함을 우선했습니다.

G. 트랜잭션 심화 (집중 대비 — 담당: 김세진, 전원 숙지)

`SubscriptionService` 의 트랜잭션 설계는 질문이 몰릴 가능성이 큼. 아래는 우리 코드 기준 정답.

Q1. `@Transactional` 은 정확히 무슨 역할을 하나요?

A. 메서드 실행을 하나의 DB 트랜잭션으로 묶습니다. 정상 종료 시 commit, 런타임 예외 시 rollback 합니다. Spring AOP 프록시가 메서드 앞뒤에 트랜잭션 begin/commit을 끼워 넣는 방식입니다.

Q2. `readOnly = true` 는 구체적으로 뭘 하나요? 성능 이점의 원리는?

A. Hibernate에 읽기 전용임을 알려 flush 모드를 수동으로 바꾸고 더티 체크(스냅샷 비교)을 생략함

니다. 영속성 컨텍스트가 엔티티 복사본을 들고 변경을 비교하는 비용이 사라져 가볍고, 조회 메서드에서 실수로 값을 바꿔도 DB에 반영되지 않아 안전합니다.

Q3. 클래스엔 `readOnly = true`, 일부 메서드엔 `@Transactional` — 우선순위는?

A. 메서드 레벨이 클래스 레벨을 덮어씁니다. 그래서 클래스 기본은 읽기 전용이고, `create` / `update` / `delete` 같은 변경 메서드에 붙인 `@Transactional` (`readOnly` 기본 `false`)이 적용돼 쓰기가 됩니다.

Q4. (핵심) `findAll` / `findById` 는 조회인데 왜 `readOnly` 가 아니라 그냥 `@Transactional` 인가요?

A. 두 메서드는 조회 시점에 갱신일 보정(`slideToNextCycle`)으로 엔티티 상태를 변경합니다. `readOnly` 면 더티 체크가 꺼져 이 변경이 저장되지 않습니다. 그래서 쓰기 트랜잭션으로 두어 지난 갱신일 보정이 DB에 반영되게 한 의도된 설계입니다.

Q5. `update()` 가 `save()` 없이 반영되는 원리(더티 체크)를 단계로?

A. ① `@Transactional` 시작 → ② `findById` 로 조회하면 영속성 컨텍스트가 스냅샷 보관 → ③ `updateDetails()` 로 필드 변경 → ④ 커밋 시 현재 상태와 스냅샷을 비교해 바뀐 컬럼만 UPDATE 자동 실행(flush). 그래서 `save()` 가 필요 없습니다.

Q6. 그럼 `update()` 에서 `@Transactional` 을 빼면 어떻게 되나요? (압박)

A. 트랜잭션·영속성 컨텍스트 경계가 사라져 조회한 엔티티가 분리(detached)되고 더티 체크가 동작하지 않아 수정이 DB에 반영되지 않습니다. 즉 변경이 조용히 사라질 수 있습니다.

Q7. 트랜잭션 경계를 왜 Service에 두나요? Controller나 Repository가 아니라?

A. 하나의 비즈니스 작업 단위 = 서비스 메서드이기 때문입니다. 여러 리파지터리 호출을 한 트랜잭션으로 묶어 원자성을 보장합니다. Repository는 단건 연산이라 경계로 부적절하고, Controller는 HTTP 관심사라 분리합니다.

Q8. 롤백은 언제 되나요? `SubscriptionNotFoundException` 은 롤백되나요?

A. 기본 규칙은 언체크 예외(RuntimeException) 시 롤백, 체크 예외는 기본 커밋입니다. `SubscriptionNotFoundException` 은 `RuntimeException` 을 상속하므로 롤백 대상입니다. 필요하면 `rollbackFor` 로 바꿀 수 있습니다.

Q9. (응용) 전파(propagation) 기본값은? `findAll()` 이 `findAll(null)` 을 부르면 트랜잭션이 2개 생기나요?

A. 기본 전파는 REQUIRED — 진행 중 트랜잭션이 있으면 합류, 없으면 새로 시작합니다. 게다가 같은 클래스 내부 호출(`findAll()` → `findAll(null)`)은 자기 호출이라 프록시를 거치지 않아 안쪽 `@Transactional` 이 새로 적용되지 않습니다. 결과적으로 트랜잭션은 하나입니다.

Q10. (응용) `@Transactional` 이 private 메서드나 같은 클래스 내부 호출엔 안 먹는 이유는?

A. 트랜잭션은 AOP 프록시로 적용되는데, 프록시는 외부에서 들어오는 호출만 가로칩니다. 같은 객체에서 `this.method()` 로 부르거나 private 메서드는 프록시를 우회합니다. (우리 코드의 `applyRenewalIfDue` 는 private지만, 이미 public 메서드의 트랜잭션 안에서 호출되므로 문제없습니다.)

- 모르면 **추측하지 말고**, "그 부분은 000이 담당했습니다 / 확인 후 답변드리겠습니다"로 넘기기.
- 답변은 **결론 먼저**, 코드 위치(**클래스명·메서드명**)를 같이 말하면 신뢰도가 올라감.
- 본인 담당 파일명은 반드시 암기: 이태경 `Subscription` / `Comment` / `GlobalExceptionHandler`, 김세진 `SubscriptionService` / `SubscriptionController` / `SubscriptionApiController`, 이상윤 `ExchangeRateService` / 테스트, 김준아 `SubscriptionView` / `HomeController`, 이예은 `*.mustache` / `style.css`.